

Práctica 07

DOCENTE	CARRERA	CURSO
DSc. Edgar Sarmiento Calisaya	Escuela Profesional de Ciencia de la Computación	Ingeniería de Software II

PRÁCTICA	TEMA	DURACIÓN
07	Rediseño	4 horas

1. Competencias del curso

- Identifica oportunidades para la evolución de software legado, y aplicar la abstracción de procedimientos, refactorización (refactoring), patrones de diseño y reutilización de software.
- Trabaja individualmente y en equipo, de manera ética y con compromiso, para el desarrollo de soluciones.

2. Competencias de la práctica

- Evolución de software legado mejorando su capacidad de entendimiento, extensión y reutilización.

3. Equipos y materiales

- Computadora
- Internet
- Material específico del curso descrito en el aula virtual:
<https://classroom.google.com/u/1/c/NzEyNjM2NTY1NzA2>

4. Objetivo

- Migrar gradualmente un proyecto con arquitectura monolítica a una arquitectura modular o microservicios utilizando el enfoque DDD, principios SOLID y el enfoque TDD (Test-driven Development); reduciendo la deuda técnica (*technical debt*), aumentando su capacidad de reutilización y previniendo la degradación de su arquitectura.

5. Prerrequisito

- Repositorio de Proyecto Final (monolito) en GitHub: ramas *master* y *development*
- IDE (VS Code, Eclipse, IntelliJ IDEA, ...)
- SonarQube IDE (Extensión/Plugin de IDE escogido): <https://www.sonarsource.com/products/sonarlint/>
- SonarQube (LTS o LTA – Community Edition 9.9.7): <https://www.sonarqube.org/downloads/>
- SonarScanner (LTS – v. 5 - 6.2.1): <https://docs.sonarsource.com/sonarqube/9.9/analyzing-source-code/scanners/sonarscanner/>

6. Actividades

- Formas **equipos** de 3 a 6 integrantes.
- El **equipo** debe:

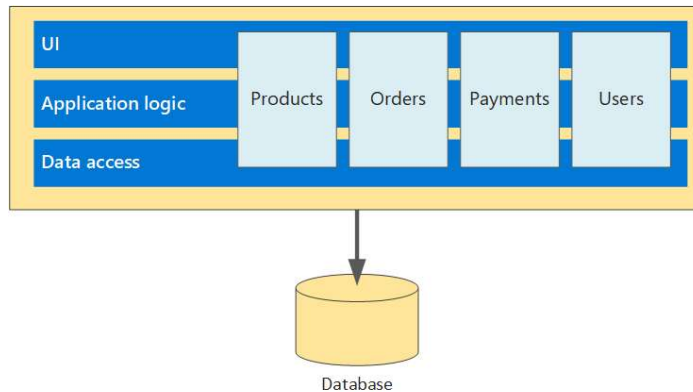
REDISEÑAR EL PROYECTO A UNA ARQUITECTURA MODULAR EN BASE A DOMAIN-DRIVEN DESIGN:
<https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis>:

- Inspeccionar la calidad del código fuente del **Proyecto Final** con la herramienta **SonarQube**

Comentado [e1]: <https://microservices.io/post/architecture/2023/02/09/assembly-architecture-definition-process.html>

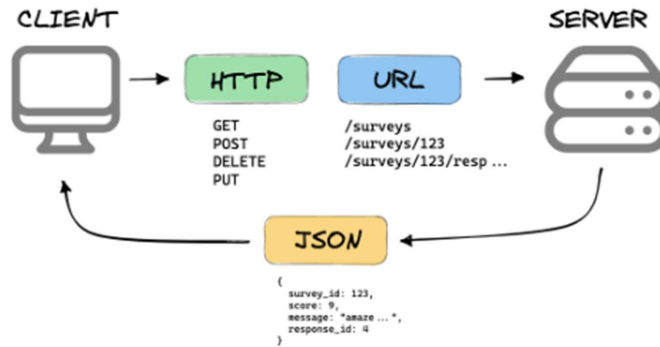
Comentado [e2R1]: 1.Discovering system operations
2.Defining subdomains
3.Designing services and their collaborations
4.Evaluating a microservice architecture
5.Refactoring a microservice architecture

- (Laboratorio 2)
2. Generar el reporte de **issues** (*code smells*, *bugs* y *vulnerabilities*) identificados por **SonarQube**.
 3. Complementar el reporte de **issues** generado, ejecutando una **revisión de código manual** (**SonarQube** no detecta todo, solo sirve de soporte) para identificar problemas que afectan el diseño y la arquitectura del **proyecto**.
 4. Entender el **lenguaje ubicuo** del proyecto: vocabulario común que es compartido por los *stakeholders*.
 5. Identificar los **módulos** (a partir de las funcionalidades – *desde el punto de vista del usuario y el negocio* – con **granularidad gruesa**) relevantes en la aplicación monolítica y aplicar el **lenguaje ubicuo**. Por ejemplo, Un módulo de administración de usuarios que agrupa componentes que gestionan las cuentas de usuarios en una tienda virtual.
 6. Definir el **modelo de dominio** de la aplicación monolítica: El modelo de dominio es un modelo abstracto del dominio del negocio.
 7. Definir **contextos delimitados** (bounded context) para los **modelos** (entidades, objetos de valor, agregados y servicios de dominio) correspondientes a los **módulos** identificados: Un **contexto delimitado** es el límite dentro de un **dominio** donde se aplica un **modelo de dominio** particular. Aplicar **límites** explícitos con **modelos y responsabilidades** claramente definidos en los **módulos** identificados. Por ejemplo, los modelos de los módulos de productos, pedidos, pagos y usuarios en una tienda virtual.



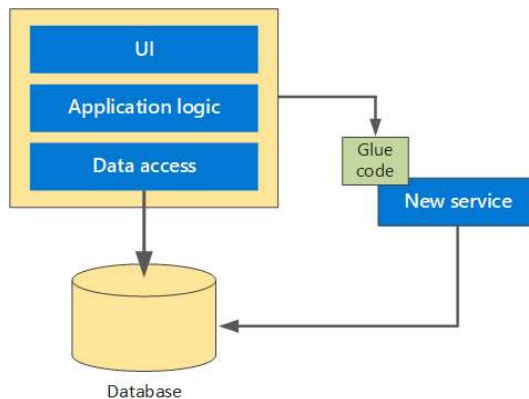
8. Identificar las **dependencias** entre los **módulos**: Los **contextos delimitados** identificados en el paso anterior son candidatos para **refactorizarlos** como **módulos**, luego en **servicios** y **microservicios**.
9. En caso de migración a microservicios: Utilizar los contextos delimitados identificados para migrarlos a microservicios en su aplicación.
10. Evaluar la **arquitectura** de la aplicación monolítica e identificar los estilos/patrones de arquitectura adecuados y consistentes con los Requisitos No Funcionales y DDD (Arquitectura en Capas): Separar las capas de presentación, lógica de negocio y persistencia (datos) de sus aplicaciones monolíticas. Módulos con su propio modelo de dominio; y capas de presentación, lógica de negocio y persistencia (datos) delimitadas.
 - **Módulos y Capas** (Presentación, Lógica de Aplicación, Lógica de Dominio, y Persistencia – infraestructura)
11. Desacoplar el **frontend del backend**:
 - Conexión **frontend-backend** a través de comunicación **API RESTful**

WHAT IS A REST API?



Fuentes: <https://medium.com/@aswathymanoop/practical-illustration-frontend-backend-connection-via-restful-api-communication-9f6930bc8e57> <https://learn.microsoft.com/en-us/dotnet/architecture/cloud-native/front-end-communication> <https://medium.com/techloop/an-introduction-to-backend-development-and-rest-apis-b1a1a978821f>

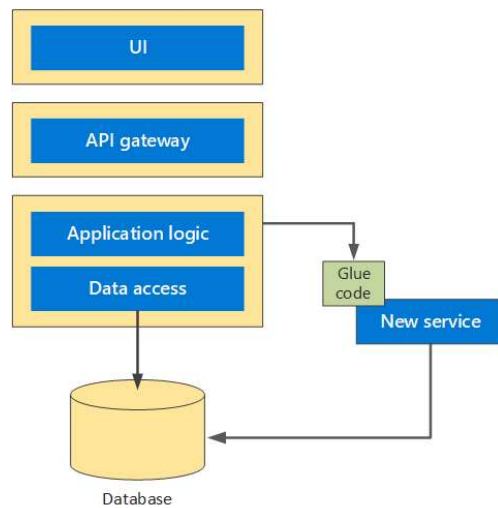
NUEVAS FUNCIONALIDADES: Caso necesite adicionar nuevas funcionalidades a su monolito, adicionarlos aplicando los principios de DDD como servicios separados.
<https://learn.microsoft.com/en-us/azure/architecture/microservices/migrate-monolith>



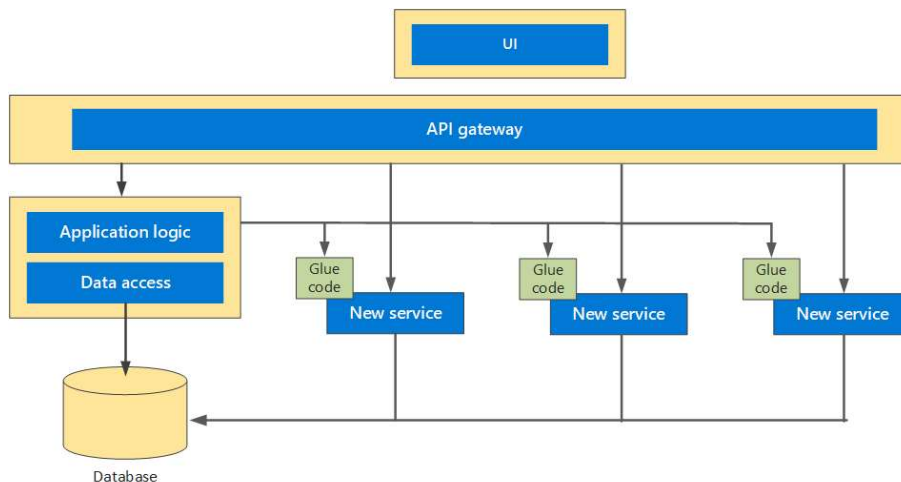
MIGRAR INCREMENTALMENTE LOS MÓDULOS A MICROSERVICIOS: <https://learn.microsoft.com/en-us/azure/architecture/microservices/migrate-monolith>

1. Determinar qué **contextos delimitados** (grupos funcionales) proporcionan el mayor valor como **microservicios** y cuáles debe **dejar el sistema** monolito por el momento.
2. Identificar los **contextos delimitados** que deben migrarse **primero**. Los **contextos delimitados con menos dependencias** que probablemente sean más fáciles de descomponer en servicios más pequeños. <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/microservice-boundaries>
3. Separar las **capas de la aplicación monolítica**: Al migrar a microservicios, a partir de los **contextos delimitados**, separar las capas de presentación, lógica de negocio y persistencia (datos) de sus aplicaciones monolíticas. *Módulos con su propio modelo de dominio; y capas de presentación, lógica negocio y persistencia (datos) delimitadas.*
4. Separa la capa de **Presentación del Backend**: Capa de presentación (UI) separada de la

- lógica de la aplicación y las capas de acceso a datos. Por medio de un **API Gateway** <https://learn.microsoft.com/en-us/azure/architecture/microservices/design/patterns>.
5. Crear **APIs (e.g., RESTful)** públicas para el microservicio identificado: Compatible con aplicaciones web y móviles. <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/microservice-boundaries>



6. [Migrar la **Data**: Base de Datos correspondiente al microservicio: <https://cloud.google.com/architecture/microservices-architecture-refactoring-monoliths?hl=es-419>]
7. [Implementar y **desplegar** el microservicio: Pueden iniciar con microservicios como módulos en la misma aplicación monolítica o el mismo servidor web]
8. Repetir los pasos 2-7 hasta completar.



DIVIDIR ESFUERZOS PARA REESTRUCTURAR Y REDISEÑAR LA ARQUITECTURA DE SOFTWARE:

9. En **GitHub Projects**, actualizar el **Proyecto**, luego:
- Crear **etapas/flujo** para seguimiento de **issues: TO-DO, CURRENT ITERATION, IN-**

PROGRESS, FIX VALIDATION y DONE:

- Crear **tareas** (nombre, descripción, responsable, etiqueta (*enhancement, redesign*), y duración) para migrar la arquitectura del proyecto.
- Asignar **tareas** a los **miembros** de los equipos responsables por su ejecución.
- Transformar **tareas** en **GitHub Issues**.

10. Referencia: <https://docs.github.com/es/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>

C. Cada **miembro** del equipo debe ejecutar sus responsabilidades (tareas asignadas):

1. **Configuración de entorno de trabajo local:**

○ **Implementación de Pruebas Unitarias y de Integración por medio de TDD o BDD:**

- Verificar que los **componentes** que deben sufrir **modificaciones** presentan pruebas automatizadas.
- Caso no existan **pruebas unitarias**, crear las **pruebas unitarias** necesarias para los **componentes** que deben sufrir **modificaciones**.
 - Diseñar casos de prueba: Valores de Prueba, Acción, Resultado Esperado
 - Implementar los scripts de prueba
 - Ejecutar los scripts de prueba
 - Reportar los resultados de la ejecución: Fallarán, y es normal.
- Organizar las pruebas unitarias de tal forma que su estructura y nombrado sea consistente con la estructura y nombrado de la implementación (Laboratorio 4).

○ **Rediseñar y refactorizar:**

- Entender las **tareas** relacionadas a los **componentes** a ser **refactorizados** y **rediseñados**.
- Modificar el código fuente base necesario.
- Re-ejecutar las **pruebas unitarias**: En caso de FALLA, volver al paso “a”, y corregir hasta que PASEN.

○ **Repositorio:**

- Subir los cambios realizados al repositorio GitHub del equipo (rama development): **commit** y **push** con nombres descriptivos (rediseño o refactoring)
 - Asociar el **commit** a el **issue** que está siendo resuelto: incluir en el mensaje **fix #<issue-number>**. Por ejemplo, “**Actualizar módulo de usuario. Fix #555**”. Referencia: <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/linking-a-pull-request-to-an-issue>
 - Buenas prácticas en **commits**: <https://midu.dev/buenas-practicas-escribir-commits-git/> <https://www.freecodecamp.org/espanol/news/como-escribir-un-buen-mensaje-de-commit/>

D. Actualizar el **estado** de las **tareas** ejecutadas (TO-DO → ... → DONE): GitHub Project.

E. Actualizar el **estado** de los **issues** (OPEN → ... → CLOSED): GitHub issues.

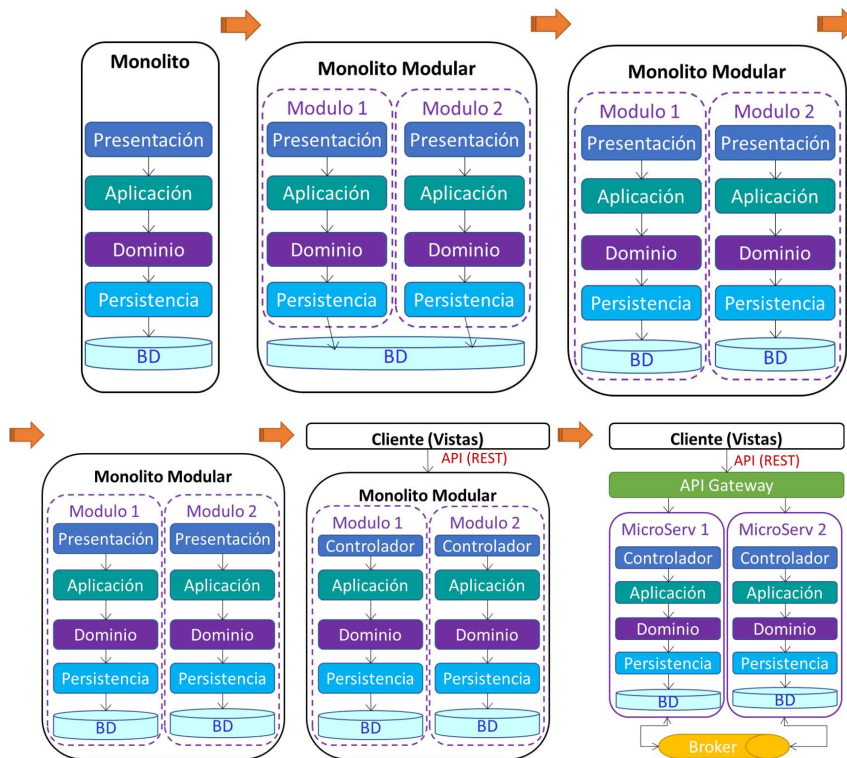
7. Entregables

- Repositorio de Proyecto Final
- Evidencia de **refactorings**: URL de **Tareas** em Github Project y **Commits**

8. Conceptos y Documentación

• **Conceptos:**

- **Domain Driven Design (DDD)** es un enfoque de desarrollo de software introducido por primera vez por Eric Evans. DDD se opone a la idea de tener un modelo único y unificado para todo el sistema. En cambio, fomenta la división del sistema en contextos delimitados, cada uno de los cuales tiene su propio modelo. DDD requiere una buena comprensión del dominio para el cual se escribirá la aplicación. El conocimiento del dominio necesario para crear la aplicación reside en las personas que la entienden: los expertos en el dominio.
- **Una arquitectura monolítica** es una aplicación con un único repositorio para sus módulos, lógica de negocio y acceso a datos. Todos los servicios se despliegan y actualizan en un solo lugar.
- **Una arquitectura modular** (modularidad) explora la creación de un programa mediante el uso de **módulos** diferentes a los de una única arquitectura monolítica. Es más como **microservicios en una arquitectura MVC**, donde todo el sistema es un conjunto de servicios independientes.
- **Una arquitectura de microservicios** distribuye sistemas como componentes independientes con funciones, recursos computacionales y bases de datos específicas. Cada componente se puede desplegar y actualizar de forma aislada, utilizando interfaces de programación de aplicaciones (API) para intercambiar datos.



- **Patterns of Modular Architecture:** <https://dzone.com/refcardz/patterns-modular-architecture#section-4>
- **Modularity in Java and .Net:** <https://dev.to/pathiknd/modularity-in-java-and-net-3dnj?ref=hackernoon.com>
- **Modularity in Java:** <https://www.baeldung.com/java-modularity>

<https://www.upgrad.com/blog/modularity-in-java/>

- **Modularity in .Net:** <https://codewithmukesh.com/blog/modular-architecture-in-aspnet-core/>
- **Modularity in Python:** <https://plainenglish.io/blog/modularization-on-python>
<https://docs.python.org/3/tutorial/modules.html>
- **Bibliografía**
 - Robert C. Martin. 2008. Clean Code: A Handbook of Agile Software Craftsmanship
 - Robert C. Martin. 2017. Clean Architecture: A Craftsman's Guide to Software Structure and Design
 - Fowler, M. (2018). Refactoring: Improving the Design of Existing Code.
 - Valente, M. T. (2024). Software Engineering: A Modern Approach. <https://softengbook.org/>
 - Newman, S. (2019). Monolith to microservices: evolutionary patterns to transform your monolith. O'Reilly Media.
 - Book Summary:
 - <https://github.com/SaraLerma/Monolith-To-Microservices>
 - <https://giulianabezerra.medium.com/padr%C3%B5es-de-migra%C3%A7%C3%A3o-de-mon%C3%B3litos-para-microservi%C3%A7os-parte-i-8b9a35251085>
 - From Monolith to Microservice – Migration Overview: <https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig>